

Week 14, Lecture 2: Instructor Script

Conditionals, Errors, and ggplot Functions

BIO/CHEM 806

Table of contents

Teaching setup	2
Segment 1: Warm-up mental model check	2
Segment 2: Functions with conditionals	2
Segment 3: Battery chemistry context	3
Segment 4: Building <code>calc_specific_capacity()</code>	3
Stage 1: naive version	3
Stage 2: <code>stop()</code>	4
Stage 3: <code>warning()</code>	4
Segment 5: <code>stop()</code> versus <code>warning()</code>	5
Segment 6: Default arguments	5
Segment 7: Molecular biology context	6
Segment 8: Wrapping <code>ggplot2</code> in a function	6
Optional synthesis: complete cycling-analysis function	7
Bridge to activity	8
Wrap-up	8
Exit ticket	8

Teaching setup

Use `week14_lecture2_student_demo.qmd` as the visible student-facing file. All instructor prompts, timing reminders, expected answers, and facilitation notes belong in this script rather than in the demo file.

The student demo is designed to be run chunk-by-chunk in Quarto/RStudio visual mode. Several chunks intentionally produce errors or warnings. Those are part of the lesson.

Segment 1: Warm-up mental model check

Display the `classify_temp()` function in the demo.

Ask students:

What does `classify_temp(15)` return, and why?

Give them about 60 seconds to think or type in chat before running the examples.

Expected answer: "moderate". The value 15 is not greater than 30 and not less than 10, so the function reaches the final `else` branch.

Follow-up if needed:

At which line does the function stop executing? Why does it not keep checking later branches?

Key point: once a branch hits `return()`, the function exits.

Segment 2: Functions with conditionals

Explain that Monday's functions did the same thing every time. Today's functions make decisions.

Emphasize:

- `if` checks the first condition.
- `else if` checks another condition only if the first was false.
- `else` catches everything that remains.
- The first true branch wins.
- `return()` exits the function immediately.

Run each example one at a time:

```
classify_temp(35)
classify_temp(5)
classify_temp(20)
classify_temp(15)
```

Before each call, ask students to predict the result.

Transition:

This early-exit pattern is the same logic we will use for error checks. If an input is bad enough, we stop before the function gets to the main calculation.

Segment 3: Battery chemistry context

Use the specific-capacity section to motivate input validation.

Read or paraphrase:

Specific capacity is measured in mAh/g. It tells us how much charge a battery electrode stores per gram of active material. The formula is simple, but the inputs are easy to get wrong.

Pause on the common research mistakes:

- wrong mass because inactive material was included;
- mass entered in milligrams instead of grams;
- zero mass entered by mistake;
- negative capacity values from instrument noise or data entry errors.

Key framing:

A silent wrong answer is worse than a loud error.

Segment 4: Building `calc_specific_capacity()`

Stage 1: naive version

Run the naive function first.

```
calc_specific_capacity <- function(capacity_mAh, mass_g) {  
  return(capacity_mAh / mass_g)  
}
```

Then run:

```
calc_specific_capacity(3.0, 0)  
calc_specific_capacity(3.0, 0.015)  
calc_specific_capacity(3.0, "graphite")
```

Instructor emphasis:

- `Inf` is dangerous because it looks like a result.
- The text input produces an error, but it is not a user-friendly error.
- The function needs to defend itself before doing the division.

Stage 2: `stop()`

Introduce `stop()` as a hard error.

Use `stop()` when the input is so wrong that the function cannot produce a meaningful result.

Run the protected version and then run the bad examples:

```
calc_specific_capacity(3.0, "graphite")  
calc_specific_capacity(3.0, 0)  
calc_specific_capacity(3.0, 0.015)
```

Expected results:

- nonnumeric mass: custom error;
- zero mass: custom error;
- valid inputs: returns 200.

Stage 3: `warning()`

Introduce `warning()` as a soft flag.

Use `warning()` when something is suspicious but still computable.

Run:

```
calc_specific_capacity(-0.5, 0.015)
calc_specific_capacity(3.0, 0.015)
```

Ask:

Why did we use `warning()` for negative capacity but `stop()` for zero mass?

Expected reasoning: negative capacity may be instrument noise worth investigating. Zero mass makes the calculation meaningless.

Segment 5: `stop()` versus `warning()`

Use the decision table in the student demo.

Walk through each row:

- wrong input type: `stop()`;
- physically impossible value: `stop()`;
- unexpected but computable value: `warning()`;
- edge of reasonable range: `warning()`.

Key rule:

Would this result mislead someone? If yes, stop. If it is suspicious but still interpretable, warn.

Segment 6: Default arguments

Explain that arguments can have default values in the function definition. The user only supplies the argument when they want something different.

Run:

```
calc_specific_capacity(3.0, 0.015)
calc_specific_capacity(3.0, 0.015, units = "Ah/g")
```

Instructor note: in this teaching example, `units` is only a label in the message. The function does not convert mAh/g to Ah/g.

Connection to familiar tools:

This is the same pattern students have used all semester in `ggplot2`. Geoms have default size, color, and alpha values. We override them only when the default is not what we need.

Segment 7: Molecular biology context

Use primer quality checks to show the same tools in a different research setting.

Emphasize:

- GC content is commonly checked for every primer.
- Standard PCR often uses a rough 40-60 percent GC target.
- Different organisms or assays may need different thresholds.
- This makes `gc_min` and `gc_max` good default arguments.

Run the three primer examples:

```
calc_primer_quality("ATGCGCATTAGC")
calc_primer_quality("GCGCGGCATGCC")
calc_primer_quality("GCGCGATATGCC", gc_min = 50, gc_max = 70)
```

Instructor note: the third example is chosen so the 50-70 percent override actually demonstrates the value of changing the defaults.

Key synthesis:

This function validates input, calls functions we already wrote, checks against thresholds, issues a warning when appropriate, and returns a named list.

Segment 8: Wrapping ggplot2 in a function

Introduce the DRY principle: do not repeat yourself.

Frame it for thesis figures:

If you are making the same style of plot repeatedly, write the plot code once and put the flexible parts in arguments.

Run `custom_theme()` first, then `plot_cycling()`.

Ask students to identify fixed and flexible parts.

Fixed parts:

- columns used for x and y;
- title and axis labels;
- basic geoms;
- overall theme.

Flexible parts:

- point size;
- font size;
- CE threshold.

Run:

```
plot_cycling(cycling_data)
plot_cycling(cycling_data, point_size = 3, ce_threshold = 98)
```

Use the advisor scenario:

Your advisor asks you to change the threshold line from 99 percent to 98 percent across 15 thesis figures. Without a function: edit 15 plot blocks. With a function: change one argument.

Optional synthesis: complete cycling-analysis function

Use this section only if time allows.

Purpose: show that a larger workflow function can validate, compute, warn, and visualize in one place.

Run:

```
analyze_cycling_data(cycling_data)
```

Key point: this is the same pattern, just one level larger.

Bridge to activity

Connect today's lecture to the Thursday activity.

Map the activity explicitly:

- `double_it()` and `exclaim_age()` review function anatomy.
- `find_max()` with conditionals reviews `if`, `else if`, and `else`.
- Adding validation to `find_max()` follows the same pattern as `calc_specific_capacity()`.
- `equal_ok = FALSE` follows the same default-argument pattern as `gc_min`, `gc_max`, and `ce_threshold`.

Wrap-up

Summarize the toolkit:

- conditionals;
- `stop()`;
- `warning()`;
- default arguments;
- functions that call other functions;
- plotting functions.

Closing line:

Write it once. Protect it with `stop()` and `warning()`. Make it flexible with default arguments. Apply it everywhere, including inside `ggplot2`.

Exit ticket

Keep this deliberately easy. The goal is retrieval practice and a quick read on what stuck, not grading.

Ask students to copy and complete two lines:

One tool I can use from today: `stop()` / `warning()` / default arguments / ggplot function

One thing I want to remember: _____